Rapport de Projet VO : Application de la discordance de Weyl 2D à la mise en correspondance dense et au tracking d'objets

Oscar Gledel, Erwann Havot

UE Vision par Ordinateur, Master II IAFA, 2025-2026, Encadrement : Alain Crouzil

1 PRINCIPE

La discordance de Weyl est une mesure mathématique permettant d'évaluer l'écart maximal d'une distribution par rapport à une répartition uniforme. Dans le contexte de la vision par ordinateur, et plus particulièrement pour la mise en correspondance dense et le tracking d'objets, elle permet de quantifier la dissimilarité entre deux régions d'image. Pour ce faire, l'article propose de calculer la variation maximale de la somme des pixels aux quatre coins de l'image.

1.1 Image intégrale

Pour éviter de recalculer la somme des pixels de manière redondante pour chaque sous-fenêtre, l'article présente une structure appelée image intégrale. L'image intégrale I_{int} d'une image d'entrée I de dimensions $L \times H$ est définie en chaque point (x, y) comme la somme de tous les pixels situés au-dessus et à gauche de ce point :

$$I_{int}(x, y) = \sum_{i=1}^{x} \sum_{j=1}^{y} I(i, j) \tag{1}$$

Cette représentation permet de calculer la somme de n'importe quelle région rectangulaire de l'image en un temps constant $\mathcal{O}(1)$, ce qui est fondamental pour garantir des performances acceptables lors du traitement de grandes images.

1.2 Discordance de Weyl : Version naïve

L'évaluation complète de la discordance de Weyl en 2D nécessite de calculer la différence maximale entre les sommes des sous-fenêtres pour les quatre orientations possibles (ancrées sur les quatre coins de l'image : Haut-Gauche, Haut-Droite, Bas-Gauche, Bas-Droite).

Pour un coin donné (par exemple le Haut-Gauche, noté P_1), la valeur de discordance est la différence entre la valeur maximale et la valeur minimale de l'image intégrale :

$$W_{P1} = \max_{x,y} \left(I_{int}(x, y)\right) - \min_{x,y} \left(I_{int}(x, y)\right) \tag{2}$$

Dans l'approche naïve, pour obtenir la discordance des trois autres coins (P_2, P_3, P_4), l'image d'entrée subit des transformations géométriques (miroir horizontal, miroir vertical, ou les deux combinés). L'image intégrale est alors recalculée de zéro pour chacune de ces images transformées. La discordance de Weyl globale W est le maximum des discordances des quatre coins :

$$W = \max(W_{P1}, W_{P2}, W_{P3}, W_{P4}) \tag{3}$$

Bien que simple à mettre en place, cette méthode requiert quatre parcours complets de l'image, entraînant une complexité non-négligeable pour une grande image.

1.3 Discordance de Weyl : Optimisation en deux passes

Pour pallier les limitations de la version naïve, l'algorithme optimisé exploite les propriétés de l'image intégrale. L'idée est qu'une seule image intégrale (calculée depuis le coin Haut-Gauche par exemple) contient déjà l'information nécessaire pour déduire les sommes des sous-fenêtres ancrées aux trois autres coins, via le principe "d'inclusion-exclusion".

En notant L la largeur et H la hauteur de l'image, les sommes rectangulaires pour les quatre coins de l'image peuvent s'exprimer uniquement à partir de I_{int} :

- **Coin 1 (Haut-Gauche)** : Valeur directe de l'image intégrale.
$$S_{P1}(x, y) = I_{int}(x, y)$$
- **Coin 2 (Haut-Droite)** : Différence avec le bord droit.
$$S_{P2}(x, y) = I_{int}(L, y) - I_{int}(x, y)$$
- **Coin 3 (Bas-Gauche)** : Différence avec le bord inférieur.
$$S_{P3}(x, y) = I_{int}(x, H) - I_{int}(x, y)$$
- **Coin 4 (Bas-Droite)**, la somme d'un bloc allant de (x, y) à (L, H) s'écrit :
$$S_{P4}(x, y) = I_{int}(L, H) - I_{int}(L, y) - I_{int}(x, H) + I_{int}(x, y)$$

Ainsi, la discordance globale est calculée en seulement deux passes (la première calculant l'image intégrale, P_1 et P_2, et la seconde calculant P_3 et P_4). Cela permet de réduire le nombre d'opérations par rapport à l'approche naïve.

2 IMPLÉMENTATION

2.1 Version naïve

Nous avons commencé par implémenter la discordance de Weyl en version naïve, afin de mieux en comprendre le concept. Les algorithmes 1 et 2 montrent le pseudo-code à l'image de notre implémentation.

Dans un premier temps nous définissons le calcul de l'intégrale comme présenté dans la section précédente, en parcourant l'ensemble de l'image et en renvoyant finalement la différence entre le maximum et le minimum des intégrales dans l'image. Il est important de noter que les images traitées sont des différences, et donc que certaines valeurs peuvent être négatives, car sinon cela reviendrait à calculer simplement la différence entre l'intégrale de toute l'image et la valeur du pixel à la coordonnée (0, 0).

Algorithme 1 Calcul de l'intégrale (Version naïve)

Entrée: Une image I de dimensions $N_x \times N_y$
Sortie : L'intégrale naïve de l'image (différence max - min)

```
V_max ← 0  V_min ← 0
for k ← 1 to N_x do
    for l ← 1 to N_y do
        S ← Σ_{i=1}^{k} Σ_{j=1}^{l} I(i, j) ;          // Somme du sous-bloc
        if S > V_max then
            V_max ← S
        else if S < V_min then
            V_min ← S
    end
end
return V_max − V_min
```

Dans un second temps, on calcule l'intégrale de l'image depuis chacun des quatre coins de cette image, et on retourne le maximum de ces 4 valeurs. Il s'agît de la définition initiale de la discordance de Weyl.

Algorithme 2 Discordance de Weyl 2D (Version naïve)

Entrée: Une image de différence I
Sortie : La valeur de discordance de Weyl W

```
W_HG ← IntegralImageNaive(I) ;                                        // Coin Haut-Gauche
W_HD ← IntegralImageNaive(MiroirHorizontal(I)) ;                 // Coin Haut-Droite (fliplr)
W_BG ← IntegralImageNaive(MiroirVertical(I)) ;                    // Coin Bas-Gauche (flipud)
W_BD ← IntegralImageNaive(MiroirHorizontal(MiroirVertical(I))) ;        // Coin Bas-Droite
return max(W_HG, W_HD, W_BG, W_BD)
```

Nous avons implémenté la fonction en Python pour cette phase de test. On constate très vite que le temps de calcul est excessivement long, avec une complexité en $O(4nm)$ si on considère une image différence de taille n x m.

2.2 Version 2-passes

Ensuite, nous avons implémenté la version de l'article, qui condense en deux passes seulement le calcul des 4 images intégrales. Le principe final reste le même, obtenir les intégrales depuis les 4 coins pour ensuite calculer le maximum parmi ces valeurs.

Algorithme 3 Discordance de Weyl 2D (Version 2-passes)

Entrée: Une image I de dimensions $L \times H$
Sortie : La valeur de discordance de Weyl W

```
// Initialisation des extrema pour les 4 quadrants
min_P1, min_P2, min_P3 ← +∞  max_P1, max_P2, max_P3 ← 0  min_P4 ← +∞, max_P4 ← −∞
// --- Passe 1 : Calcul de l'image intégrale, P1 et P2 ---
for y ← 1 to H do
    SommeLigne ← 0  min_loc ← +∞, max_loc ← 0
    for x ← 1 to L do
        SommeLigne ← SommeLigne + I(x, y)  I_int(x, y) ← I_int(x, y − 1) + SommeLigne ; // Image intégrale
        min_loc ← min(min_loc, I_int(x, y))  max_loc ← max(max_loc, I_int(x, y))
    end
    min_P1 ← min(min_P1, min_loc)  max_P1 ← max(max_P1, max_loc)
    C_P2 ← I_int(L, y) ;                                                    // Bord droit
    min_P2 ← min(min_P2, C_P2, C_P2 − max_loc)  max_P2 ← max(max_P2, C_P2, C_P2 − min_loc)
end
// --- Passe 2 : Calcul pour P3 et P4 ---
for x ← 1 to L do
    min_loc ← +∞, max_loc ← 0  max_H ← I_int(x, H) ;                   // Bord inférieur
    for y ← 1 to H do
        min_loc ← min(min_loc, I_int(x, y))  max_loc ← max(max_loc, I_int(x, y))
        max_L ← I_int(L, y) ;                                               // Bord droit
        val_P4 ← I_int(x, y) − max_L − max_H  min_P4 ← min(min_P4, val_P4)  max_P4 ← max(max_P4, val_P4)
    end
    C_P3 ← I_int(x, H) ;                                               // Bord inférieur
    min_P3 ← min(min_P3, C_P3, C_P3 − max_loc)  max_P3 ← max(max_P3, C_P3, C_P3 − min_loc)
end
// --- Résultat : Maximum des différences ---
W_1 ← max_P1 − min_P1  W_2 ← max_P2 − min_P2  W_3 ← max_P3 − min_P3  W_4 ← max_P4 − min_P4
return max(W_1, W_2, W_3, W_4)
```

Le principe de l'algorithme est décrit en pseudo-code dans l'algorithme 3. Cela consiste à effectuer deux passes différentes sur

les données. La première permet de calculer $P1$ et $P2$ qui sont indépendants, puis la seconde calcule $P3$ et $P4$ qui dépendent des valeurs de $P1$ calculés dans la passe précédente.

En terme de code, nous avons développé cette méthode en python à la suite du développement de la version naïve, puis nous l'avons implémenté en C++ afin de gagner en temps d'exécution. Un résumé des performances des différents algorithmes est présenté dans la section 2.4.

2.3 Version de l'article

Dans l'article, les auteurs mettent en avant l'utilisation de directives AVX pour le traitement des entiers 8 par 8. (données stockés sur 256 bits concurrents en mémoire). Nous avons donc implémenté une version utilisant les directives AVX. Une partie de cette implémentation est présentée dans le Listing 1.

```
int x = 0;
for(; x <= width - 8; x += 8)
{
    unsigned int current = y * width + x;

    // Loading 8 next image pixels
    __m128i pixels_8bit = _mm_loadl_epi64(reinterpret_cast<const __m128i*>(&image.Img[current]));
    __m256i vec = _mm256_cvtepu8_epi32(pixels_8bit);

    // Next 4 blocks : calculating integral using paper method
    vec = _mm256_add_epi32(vec, _mm256_slli_si256(vec, 4));
    vec = _mm256_add_epi32(vec, _mm256_slli_si256(vec, 8));

    __m256i shift_lane = _mm256_permute2x128_si256(vec, vec, 0x08);
    __m256i broadcast_high = _mm256_shuffle_epi32(shift_lane, 0xFF);
    vec = _mm256_add_epi32(vec, broadcast_high);

    __m256i rowSum_vec = _mm256_set1_epi32(rowSum);
    vec = _mm256_add_epi32(vec, rowSum_vec);

    __m256i vec_above = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(&integralImage[
    current - width]));
    __m256i integral_vec = _mm256_add_epi32(vec, vec_above);

    // Storing values in integral image
    _mm256_storeu_si256(reinterpret_cast<__m256i*>(&integralImage[current]), integral_vec);

    // Computing locals min/max (8 differents locals min/max)
    localMinP1_vec = _mm256_min_epu32(localMinP1_vec, integral_vec);
    localMaxP1_vec = _mm256_max_epu32(localMaxP1_vec, integral_vec);

    __m128i top_half = _mm256_extracti128_si256(vec, 1);
    rowSum = static_cast<uint32_t>(_mm_cvtsi128_si32(_mm_shuffle_epi32(top_half, 0xFF)));
}
```

Listing 1: Extrait de l'optimisation du calcul de l'image intégrale avec les instructions vectorielles AVX2

Pour résumer le principe de l'utilisation des instructions AVX, cela consiste à exécuter une série précise d'opérations sur les 8 entiers alignés en mémoire. Ces instructions sont les suivantes (dans l'ordre) : ADD, SHIFT, ADD, en tenant compote du fait que les données ont été shiftés dès le départ en input (autrement on aurait SHIFT, ADD, SHIFT, ADD).

Il faut noter que le shift sur un aligment mémoire 4 données ne pose pas de problème, mais sur 8 données on tombe toujours sur deux blocs mémoires. Ainsi, on doit utiliser le High broadcast pour contourner cette spécificité matérielle.

2.4 Comparaison des performances

Dans cette section, nous comparerons les performances des différents programmes (Naïf, 2-passes et AVX-optimized) dans les différents langages (C++ et Python). La machine sur laquelle sont mesurées ces performances utilise un Intel® Core™ i5-10210U × 8 cores, avec 8 GB de mémoire vive. Il est possible que le speed-up varie en fonction de la machine utilisée.

[H]

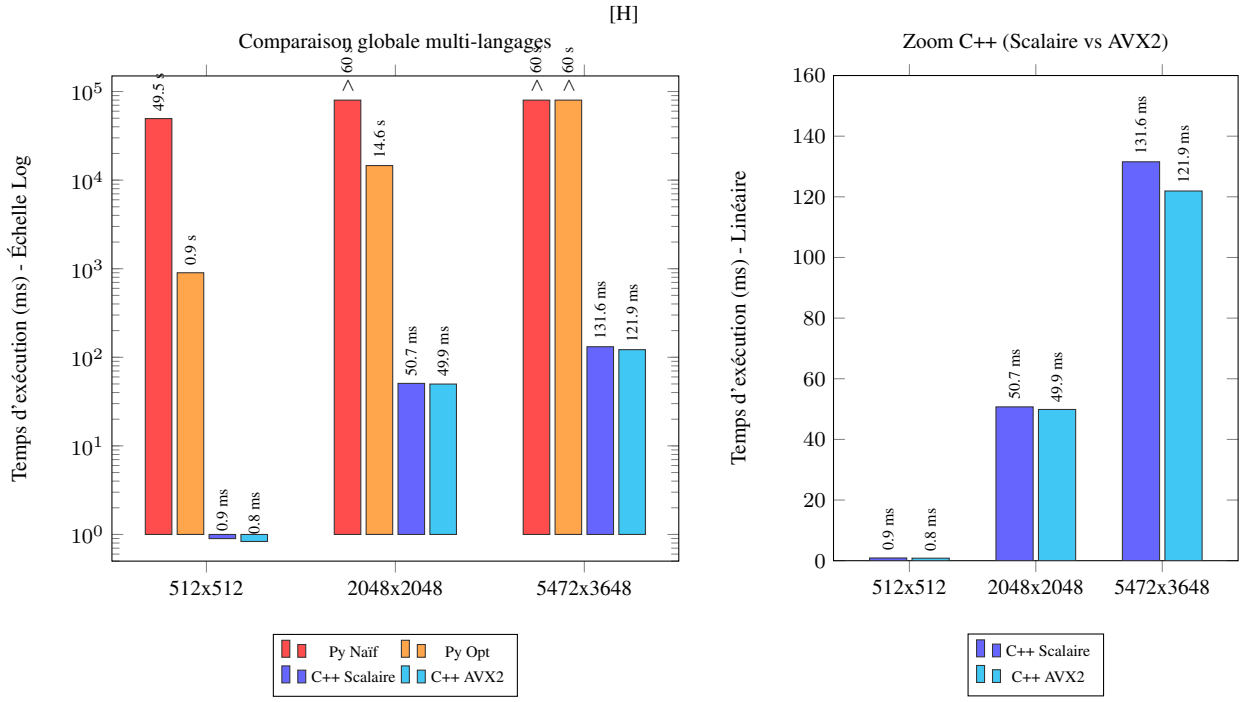

Figure 1. Comparaison des temps de calcul de la discordance de Weyl. **À gauche :** Évaluation globale mettant en évidence le gain de l'approche optimisée (échelle logarithmique). **À droite :** Zoom sur l'implémentation C++ montrant l'accélération matérielle apportée par les instructions vectorielles AVX2 (échelle linéaire).

La Figure 1 montre les temps d'exécutions moyens en fonction de la version de l'algorithme. On voit clairement l'écart entre le C++ et le Python, qui est logique car notre code python est interprété plutôt que compilé, et exécuté dans un notebook.

Le point d'intérêt de cette figure est plutôt du côté droit, avec les temps moyens pour les versions C++ Scalaires et AVX qui n'ont que peu de différence (on note un facteur d'accélération d'environ 1.07). Cette faible différence s'explique par trois éléments. D'abord, la compilation s'effectue avec le flag $-O3$, ce qui laisse le droit au compilateur d'ajouter des instructions AVX sur la fonction scalaire. Ensuite, le code évalué contient une partie scalaire, même dans le code AVX. Cette partie non négligeable peut limiter le facteur que l'on observe. Enfin, le calcul des images intégrales comporte beaucoup d'accès à la mémoire RAM, ce qui est un goulot d'étranglement comparé à l'exécution d'instructions sur le CPU.

3 APPLICATION À LA MISE EN CORRESPONDANCE DENSE

3.1 Principe

L'application de la discordance de Weyl à la mise en correspondance dense s'effectue en faisant une recherche du pixel le plus ressemblant par rapport à un autre pixel dans la deuxième image. Nous devons définir deux hyper-paramètres : la taille des patchs et la distance max entre les coordonnées de pixels correspondants. De plus, on fait deux suppositions pour simplifier les calculs : les images en entrée sont rectifiées (ce qui nous permettra de ne regarder les candidats potentiels que sur la même ligne dans l'autre image), et les voisins d'un point dans l'image gauche (resp. droite) se trouvent forcément sur la droite de ce pixel (resp. gauche). Ainsi, l'algorithme consiste à rechercher parmi les x pixels adjacents dans l'image correspondante celui qui ressemble le plus au pixel actuel. Étant donné que ça n'a pas de sens sur un seul pixel, on utilise plutôt un "voisinage" autour du pixel pour calculer la discordance de Weyl.

On sauvegarde finalement, pour chaque pixel de l'image gauche (resp. droite), la distance en nombre de pixels avec le pixel le plus ressemblant sur la même ligne dans l'image droite (resp. gauche) à une distance limite de $maxdiscrepancy$. On obtient deux cartes de disparités, une pour l'image gauche et une pour l'image droite.

3.2 Données de test

Pour les tests de cette application, nous utiliserons les datasets Middleburry de 2001, 2003 et 2005. Pour chaque jeu de donnée, on a : une image gauche rectifiée, une image droite rectifiée, une carte de disparité gauche servant de vérité terrain, une carte de disparité gauche servant de vérité terrain. La figure 2 montre un exemple des images de ce dataset.

Figure 2. Extrait du dataset Middlebury-2003, image "Cones". De gauche à droite, on retrouve : image gauche rectifiée, image droite rectifiée, référence de disparité gauche, référence de disparité droite.

Pour ce qui est du dataset de 2005, nous avons pris les images en mode 'third', ce qui veut dire que l'image fait un tier de sa taille originale, pour des raisons de temps de calcul.

3.3 Résultats

La figure 3 montre les résultats de l'application de la discordance de Weyl à la mise en correspondance dense de pixels. Visuellement, on peut très facilement distinguer les motifs de l'image de référence. La dernière colonne de la figure met en évidence l'application de l'algorithme LRCC sur nos cartes de disparités, qui consiste à vérifier que les deux images renvoient bien la même disparité pour deux pixels associés. Si ce n'est pas le cas, l'algorithme fixe la disparité à 0 pour dire que les pixels n'ont pas de correspondant. Cela met bien en évidence les zones où l'algorithme n'est pas sûr de lui et s'est donc trompé.

La figure 4 nous montre que les résultats ne sont pas excellent, avec un Bad Pixel Rate (BPR) qui dépasse quasiment tout le temps les 0.5. Ce BPR est calculé avec une tolérance de 1, on peut donc en partie expliquer cette forte valeur par des erreurs légèrement supérieures à 1 en moyenne. Cela se confirme avec la MSE, qui elle se être relativement basse et donc plus représentative de nos résultats.

4 APPLICATION AU TRACKING D'OBJETS

4.1 Principe

L'application de la discordance de Weyl dans le tracking s'effectue avec une première étape de création d'un patch de référence. Pour cela, nous avons créé un script permettant d'afficher la première image (frame) d'une vidéo sur laquelle nous avons dessiné un rectangle représentant la bounding box (ou boite englobante en français) autour de l'élément que nous souhaitions suivre au cours de la vidéo. La détermination de la bounding box s'effectue manuellement. La seconde étape consiste à lire la vidéo et appliquer frame par frame la méthode suivante : on découpe l'image à l'endroit où le patch est passé, on effectue le calcul de la discordance de Weyl sur la différence entre l'image croppée et le patch et on garde en mémoire la valeur obtenue. A la fin nous obtenons une carte de valeurs (heatmap) sur laquelle nous récupérons le minimum qui représente, selon le calcul, la zone la plus probable où se trouve l'objet du patch. Puis en fonction de ce résultat nous dessinons le rectangle correspondant sur l'image. Cette technique pourrait se comparer à la convolution.

4.2 Données de test

Les données que nous avons utilisées lors de nos tests sont issues du dataset ToCaDa de l'IRIT. Nous pouvons y retrouver différentes vidéo obtenue via des caméra placées autour du bâtiment principal. Nous pouvons y observer le passage de personnes et des déplacement de véhicules (majoritairement des voitures). Durant ce projet nous nous sommes intéressés à leur suivi. Nous avons commencé sélectionner des vidéos et nous les avons coupée pour en obtenir des extraits ne dépassant pas 15 secondes. Nous avons

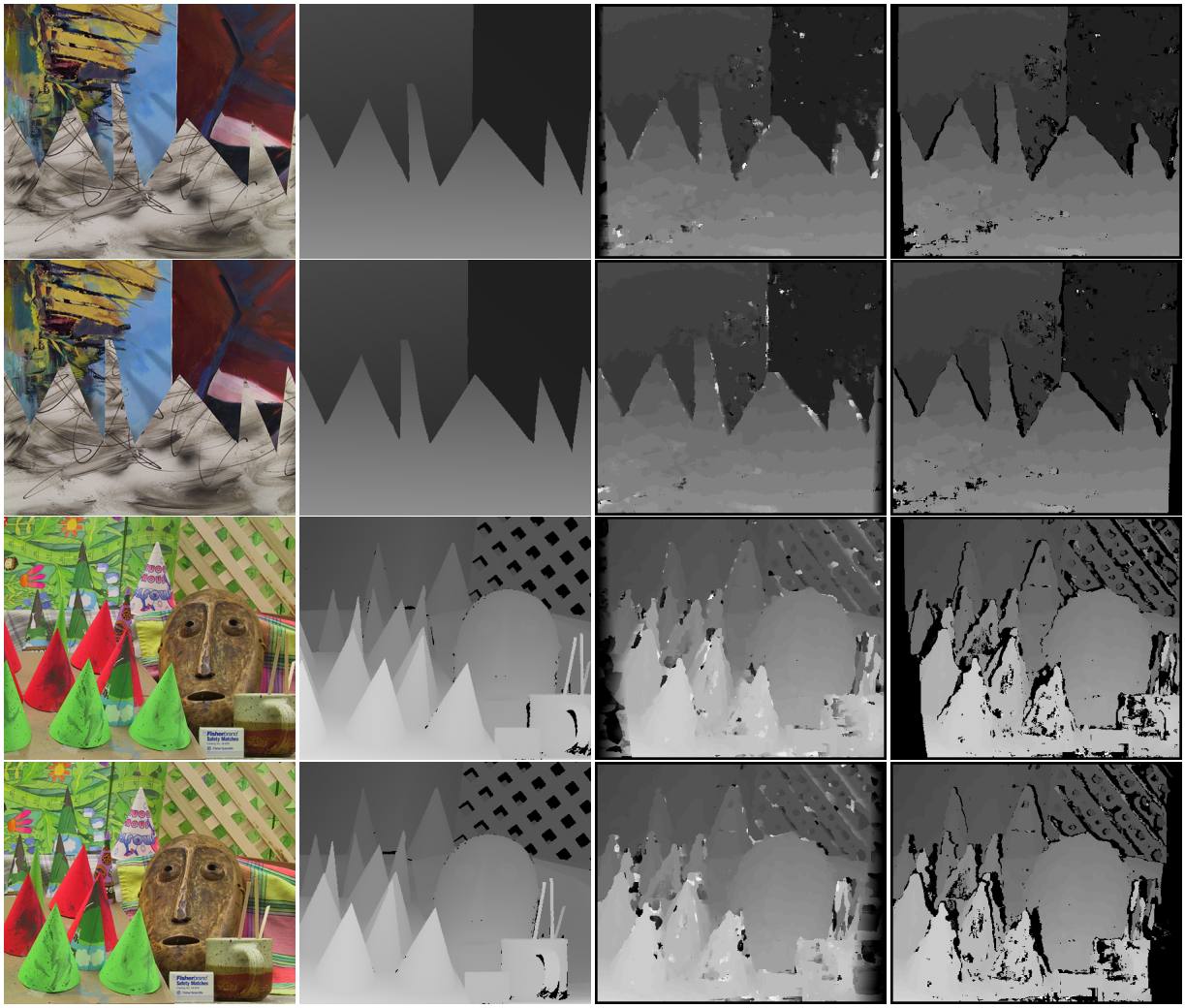

Figure 3. Résultats obtenus sur les images 'Sawtooth' et 'Cones' du dataset Middlebury. Les lignes correspondent, dans l'ordre, à : Sawtooth left, Sawtooth right, Cones left, Cones right. Pour les colonnes, on retrouve à chaque fois, dans l'ordre : image originale, carte des disparités de référence, carte des disparités obtenue par la méthode de Weyl, et carte des disparités rectifiée par l'algorithme Right-Left Consistency Check (LRCC).

préparé 5 extraits provenant de 5 vidéos avec des caméras et des angles de vue différents. Les vidéos d'origines utilisées sont celles en basse résolution pour des raisons d'efficacité de traitement. Pour chaque vidéo, nous avons 2 types de données, chacune étant utilisée pour l'exécution en fonction du langage du code. Pour le code en python, nous utilisons directement les fichiers en ".mp4" et une lecture avec l'utilisation de la bibliothèque openCV. Pour le code en C++, nous avons enregistré chaque frame au format ".jpeg" via un script python, pour éviter l'installation locale de la librairie openCV qui aurait demandé beaucoup de temps.

4.3 Résultats

Pour pouvoir analyser nos résultats sur les images nous avons utilisé les détections du modèle "nano" de YOLOv11 développé par Ultralytics sur notre objet de référence. Le principe repose sur l'enregistrement de la détection de l'objet de référence par le modèle dans un fichier ".txt" pour chaque frame. Côté C++ et Weyl, nous récupérons un fichier ".csv" avec ces mêmes informations. Puis via un script python nous écrivons sur la vidéo le résultat des deux fichier à chaque frame. Nous pouvons retrouver un exemple dans la figure 5.

Pour la vérification des résultats obtenus nous enregistrons toutes les images de disparité du programme ce qui permet de

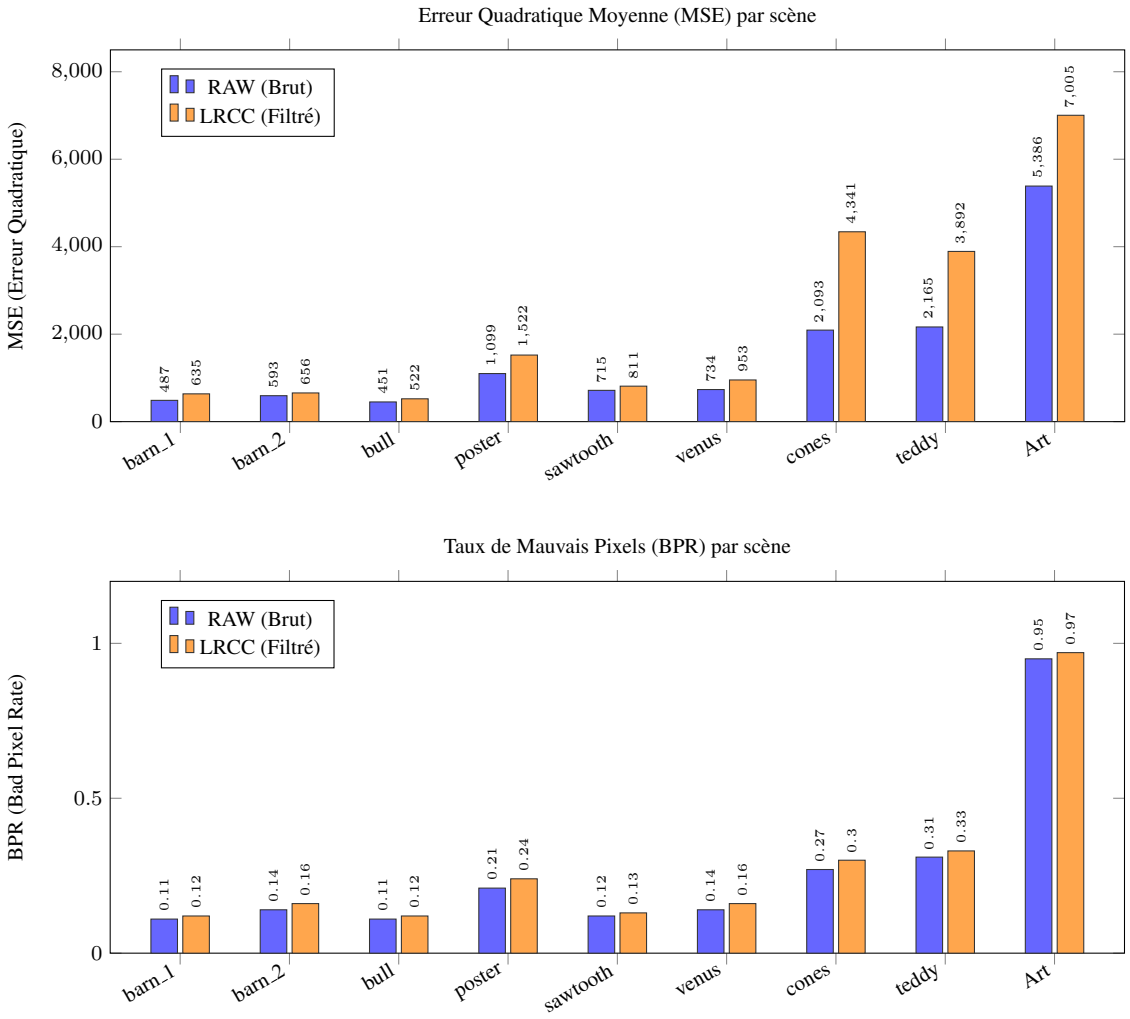

Figure 4. Évaluation quantitative de la mise en correspondance dense sur les datasets Middlebury. Les scores présentés sont la moyenne des images de disparité gauche et droite pour chaque scène. On observe de bonnes performances générales de l'algorithme brut (entre 11% et 31% de BPR). Cependant, l'application du filtre de cohérence gauche-droite (LRCC) dégrade légèrement les métriques numériques, ce qui s'explique par l'invalidation des pixels dans les zones d'occlusion. Le dataset "Art", présentant des caractéristiques d'échelle de disparité très différentes, obtient des scores d'erreur majeurs révélant la nécessité d'adapter le paramètre de disparité maximale selon la résolution.

vérifier le bon fonctionnement de la recherche la concordance de nos boîtes finales. Nous pouvons observer un exemple sur la figure 6

Pour compléter les résultats nous effectuons une mesure de type Mean Squarred Error (MSE) entre Yolo qui détient le résultat que l'on considère vérité terrain et le retour de notre programme. Nous ajoutons aussi une mesure de l'Intersection over Union (IoU) permettant de savoir en plus de la mesure précédente si notre résultat touche ou non la zone référence et à quel pourcentage. Nous pouvons voir dans la figure 7 les différentes mesures.

5 CONCLUSION

5.1 Limites de la méthode

Les limites de l'utilisation de la discordance de Weyl sont assez visibles dans les résultats de la mise en correspondance. En effet, on peut voir au travers de la MSE et du BPR que cette méthode peine à détecter des éléments qui ne font pas partie d'un pattern clairement identifiable. Tout comme pour le tracking, les effets de distorsions dû à la profondeur faussent beaucoup les résultats.

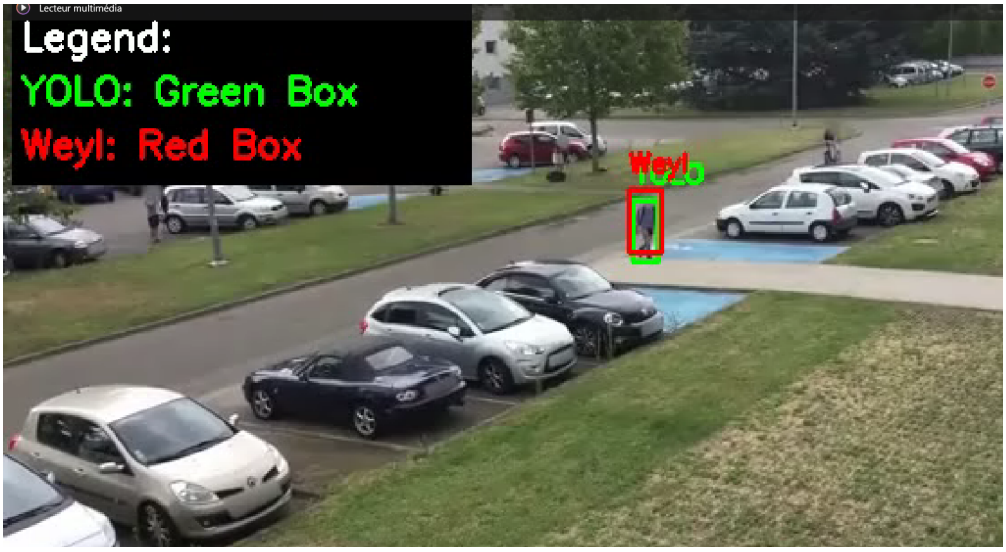

Figure 5. Exemple d'un image d'une vidéo affichant les boîtes englobantes de Yolo et de Weyl.

Figure 6. A gauche l'image finale obtenue avec le rectangle obtenue après exécution du programme. A droite l'image de disparité

Dans son implémentation actuelle, la méthode de Weyl montre des limites pour le suivi d'objets (tracking) en conditions complexes. Lors de nos différents essais, nous avons pu remarquer qu'un changement d'orientation ou d'échelle (taille) de l'objet affectait considérablement les performances de la méthode. Toutefois, si la cible conserve la même apparence globale et le même axe tout au long de la séquence vidéo, le suivi s'effectue de manière correcte et robuste.

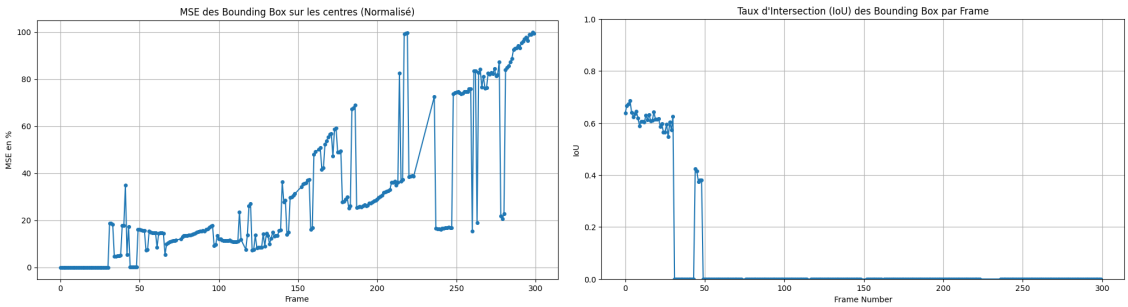

Figure 7. Courbes de MSE et d'IoU avec les frames en abscisse et les resultats des métricques en ordonnées.

5.2 Améliorations possibles

Si l'on souhaite améliorer la reconstruction des cartes de discordance, une idée que nous n'avons pas implémenté est de choisir plus finement la taille du patch selon les images, en approchant par exemple par tâtonnement. Une autre amélioration pourrait consister à estimer une valeur intermédiaire/lissée dans les zones où la détection s'est mal passée.

Afin d'améliorer les performances et la robustesse du suivi, plusieurs perspectives peuvent être envisagées. Tout d'abord, la définition de la zone de recherche (actuellement une marge fixe) pourrait être rendue dynamique grâce à un calcul de flux optique, permettant d'estimer le vecteur de mouvement de la cible d'une image à l'autre. Par ailleurs, l'intégration d'un modèle prédictif tel que le filtre de Kalman constituerait une évolution majeure. En modélisant la cinématique de l'objet (position, vitesse), un tel filtre permettrait de guider la recherche de la mesure de Weyl, réduisant les faux positifs et maintenant le suivi même lorsque l'objet est temporairement masqué.

6 LIENS, RÉFÉRENCES ET ÉLÉMENTS DE CODE

Les sources du projet sont disponibles sur GitHub à l'adresse suivante : https://github.com/Oscosc/Projet-VO-2026

6.1 Références

Christian Motz et Bernhard A. Moser, "On a Fast Implementation of a 2D-Variant of Weyl's Discrepancy Measure", publié dans les actes du OAGM-ARW Joint Workshop, 2016.

Gabriele Facciolo, Nicolas Limare et Enric Meinhardt-Llopis, "Integral Images for Block Matching", publié dans la revue Image Processing On Line (IPOL), vol. 4, pp. 344–369, 2014.